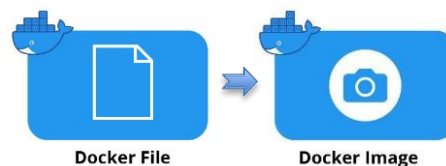


Docker Images erstellen

App Ref. Card 01 und 02



Beschreibung

Dieses Dokument dient als ersten Einstieg in die Erstellung von Docker-Images mit Docker-Files. Hierfür werden die App Ref. Card 01 und Card 02 als Beispiel und Ausgangslage verwendet.

Inhaltsverzeichnis

1	Docker Images erstellen	2
1.1	Kompetenzen	2
1.2	Einleitung.....	2
1.3	Aufgabe 1 - Docker-Image für App Ref. Card 01 (Spring Boot) erstellen	2
1.3.1	<i>Docker-Image benennen</i>	<i>3</i>
1.3.2	<i>Build-Befehl und Tag in einem</i>	<i>4</i>
1.3.3	<i>Container aus Docker-Image starten.....</i>	<i>4</i>
1.3.4	<i>Reflexion.....</i>	<i>4</i>
1.4	Aufgabe 2 - Dockerfile App Ref. Card 02 (React App) erstellen	5
2	Nice to know	5
2.1	Befehle	5

1 Docker Images erstellen

1.1 Kompetenzen

- Ich kann ein Dockerfile für eine einfache App erstellen.
- Ich kann ein Basis-Image für einen App-Typ (Spring Boot, React) auswählen
- Ich kann je nach App-Typ (Spring Boot, React) die Befehle im File bestimmen.
- Ich kann aus einem Image einen Container starten.
- Ich kann die Images und Container auf meinem Computer verwalten.

1.2 Einleitung

Ein Dockerfile ist eine Textdatei, die die Schritte zur Erstellung eines Docker-Images beschreibt. Es enthält Befehle, die angeben, welche Aktionen bei der Erstellung des Images durchgeführt werden sollen.

In dieser Anleitung wollen wir ein eigene Docker-Images von Apps erstellen. Die folgende Grafik zeigt den groben Ablauf bei der Erstellung eigener Images.

1. Der Aufbau des Images wird in einer Datei (Dockerfile) beschrieben.
2. Das Image wird mit dem Build-Befehl erstellt.
3. Danach kann liegt das erstellte Image im lokalen Repository, aus welchem Container erstellt werden.



1.3 Aufgabe 1 - Docker-Image für App Ref. Card 01 (Spring Boot) erstellen

In diesem ersten Beispiel soll ein Docker-Image von einer Spring-Boot App erstellt werden. Hierfür wird ein Dockerfile erstellt und ein Basis-Image gewählt, auf welchem das eigene Image aufgebaut werden soll.

Die App findet sich in folgendem Git-Repository:

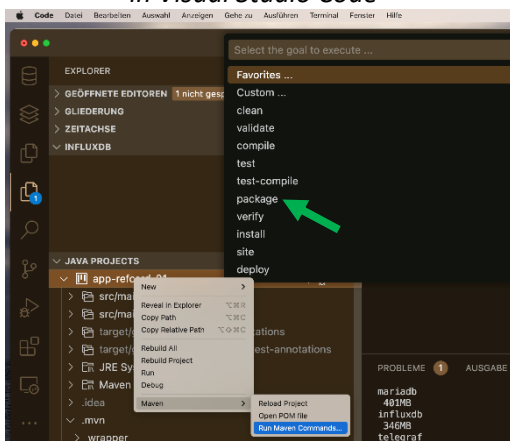
<https://gitlab.com/bbwrl/m347-ref-card-01>



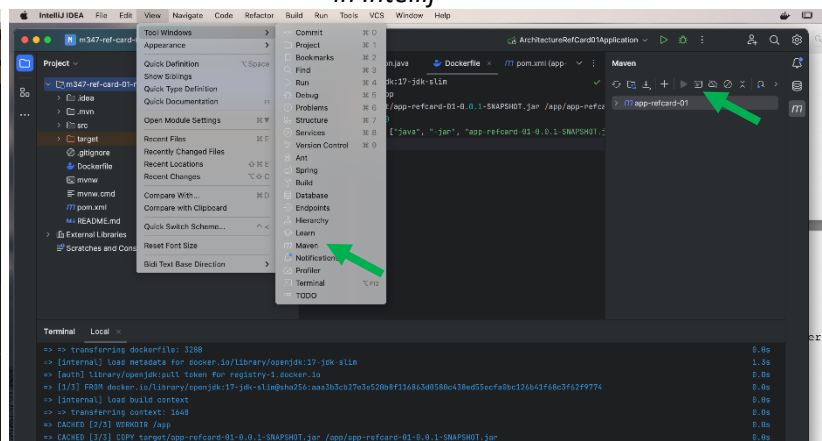
a) Klonen Sie die App auf Ihren Computer

b) Das Projekt mit Maven kompilieren, Packages hinzufügen und in ein ausführbares Format (jar) packen:
`mvn package`

in Visual Studio Code



in IntelliJ



c) Erstellen Sie im Ordner des Projektes eine Datei mit dem Namen «Dockerfile» und bearbeiten Sie die Datei wie folgt:

1. Wählen eines Basis-Images (in unserem Fall openjdk:17), welches die benötigte Java Version enthält.
2. Definieren des Arbeitsverzeichnis (WORKDIR)
3. Spring Boot Anwendung (Artefakt) in das Arbeitsverzeichnis kopieren.
4. Anwendung mittels des ENTRYPOINT Befehls starten.

Das Dockerfile für die App Ref. Card 01 könnte z.B. so aussehen:

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/app-refcard-01-0.0.1-SNAPSHOT.jar /app/app-refcard-01-0.0.1-SNAPSHOT.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app-refcard-01-0.0.1-SNAPSHOT.jar"]
```

- In diesem Beispiel wird das offizielle openjdk-Image mit Java 17 als Basisimage verwendet. Der Arbeitsbereich innerhalb des Containers wird auf den Ordner "/app" festgelegt. Dann wird die Anwendungs- JAR-Datei mit dem Namen "app-refcard-01-0.0.1-SNAPSHOT.jar" und die Abhängigkeiten in den Container kopiert. Der Port 8080 wird für den Container freigegeben und der Startbefehl für den Start des Containers wird festgelegt.

Das Dockerfile muss immer mit dem Namen "Dockerfile" oder "dockerfile" benannt werden (ohne Dateiergung). Es ist wichtig, dass der Dateiname genauso geschrieben wird.

Das mit dem Dockerfile definierte Image kann nun mit folgendem Befehl gebaut werden.

```
$ docker build .
```

Wenn das Dockerfile in einem Unterverzeichnis des aktuellen Verzeichnisses gespeichert wäre, müssen Sie dem Pfad zu diesem Unterverzeichnis dem Build-Befehl anhängen.

```
$ docker build -f /path/to/Dockerfile .
```

Der "-f" Parameter ist optional und wird normalerweise nicht benötigt, wenn das Dockerfile im aktuellen Verzeichnis gespeichert ist und den Standardnamen "Dockerfile" oder "dockerfile" hat.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<none>	<none>	8c5c742644c8	14 hours ago	421MB
httpd	latest	dc0ffcb6f5c2	11 days ago	139MB
nginx	latest	d8906c7d4c44	2 weeks ago	135MB
hello-world	latest	46331d942d63	10 months ago	9.14kB

1.3.1 Docker-Image benennen

Um den Namen des Repositories und des Tags eines Docker-Images zu setzen, kann der Befehl "docker tag" verwendet werden. Alternativ dazu kann der dies mit der -t Option direkt dem Build hinzugefügt werden.

Der Befehl hat folgendes Format:

```
$ docker tag [Image ID] [Repository Name]:[Tag]
```

Beispielsweise, um das Image mit der ID "8c5c742644c8" den Namen "app-refcard-01" und den Tag "1.0" zu geben, würde der Befehl so aussehen:

```
$ docker tag 8c5c742644c8 app-refcard-01:1.0
```

Dann können Sie es mit "docker images" verifizieren, dass die Repository und Tag Spalten jetzt entsprechende Namen haben.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
app-refcard-01	1.0	dc0ffcb6f5c2	11 days ago	139MB
httpd	latest	dc0ffcb6f5c2	11 days ago	139MB
nginx	latest	d8906c7d4c44	2 weeks ago	135MB
hello-world	latest	46331d942d63	10 months ago	9.14kB

1.3.2 Build-Befehl und Tag in einem

```
$ docker build -t [Repository Name]:[Tag] .
```

1.3.3 Container aus Docker-Image starten

Um einen Docker-Container aus einem Image zu starten, kann der Befehl "docker run" verwendet werden. Der Befehl hat folgendes Format:

```
$ docker run [options] [image name or ID]
```

Ein Beispiel wäre:

```
$ docker run -d -p 8080:8080 app-refcard-01:1.0
```

-d Option lässt den Container im Hintergrund laufen.

-p Option bindet einen Port des Hosts mit einem Port des Containers.

Der Befehl startet den Container aus dem Image "app-refcard-01:1.0" und bindet den internen Port 8080 an den Port 8080 des Hosts. Der Container läuft im dann Hintergrund.

1.3.4 Reflexion

- Beschreiben Sie jeweils oberhalb des Befehls möglichst genau, was der folgende Befehl bewirkt.
- Markieren Sie im Feld rechts mit einem X, welche Befehle direkt mit dem Projekt-Typ Spring-Boot zu tun haben.

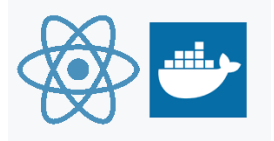
FROM openjdk:17-jdk-slim	
WORKDIR /app	
COPY target/app.jar /app/app.jar	
EXPOSE 8080	
ENTRYPOINT ["java", "-jar", "app.jar"]	

1.4 Aufgabe 2 - Dockerfile App Ref. Card 02 (React App) erstellen

In diesem ersten Beispiel soll ein Docker-Image von einer React App erstellt werden. Hierfür wird ein Dockerfile erstellt und ein Basis-Image gewählt, auf welchem das eigene Image aufgebaut werden soll.

Die App findet sich in folgendem Git-Repository:

<https://gitlab.com/bbwrl/m347-ref-card-02>



Wenn Sie ein Dockerfile für eine React-Anwendung erstellen, gibt es, wie bei der Spring-Boot App ein paar grundlegende Schritte, die beachtet werden sollten:

1. Base Image wählen (z.B. node:16.19...)
2. Definieren des Arbeitsordners (WORKDIR)
3. Kopieren der-Anwendung in den Arbeitsordner (COPY)
4. Installieren der benötigten Abhängigkeiten (RUN npm install)
5. Definieren des verfügbaren Ports (EXPOSE 3000)
6. Definieren des Start-Skripts (ENTRYPOINT ["npm", "start"])

```
# Use an official node.js image as the base image
FROM node:16.19.0-bullseye-slim

# Set the working directory in the image
WORKDIR /app

# Copy all files to the image
COPY . .

# Install the npm dependencies
RUN npm ci

# Expose port 3000 to the host
EXPOSE 3000

# Specify the command to run when the container starts
ENTRYPOINT [ "npm", "start" ]
```

Wie auch bei der ersten App wird das Image mit folgendem Befehl erstellt.

```
$ docker build .
```

Das neue Docker-Image kann nun auch mit dem Tag benannt werden.

```
$ docker tag {id} app-refcard-02:1.0
```

Start des Containers (aus dem Image), inkl. Bindung des internen Ports 3000 an den Port 3000 des Hosts:

```
$ docker run -d -p 3000:3000 app-refcard-02:1.0
```

2 Nice to know

2.1 Befehle

Der Unterschied, zwischen dem CMD- und dem ENTRYPOINT-Befehl, liegt darin, dass der ENTRYPOINT-Befehl für den Start des Containers verantwortlich ist und von aussen nicht überschrieben werden kann. CMD hingegen definiert Befehle, die beim Start des Containers standardmässig ausgeführt werden, aber durch übergebene Befehle beim Starten des Containers überschrieben werden können. Daher kann es in bestimmten Fällen vorkommen, dass ein ENTRYPOINT-Befehl den Container erfolgreich startet, währenddessen der CMD-Befehl nicht funktioniert.